

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №10.

Тема: «»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

10	Задача 1 1. Контейнер - вектор 2. Тип элементов Money (см. лабораторную работу №3). Задача 2 Адаптер контейнера – очередь с приоритетами. Задача 3 Ассоциативный контейнер – множество с дубликатами		
	Задание 3	Задание 4	Задание 5
	Найти минимальный элемент и добавить его на заданную позицию контейнера	Найти элементы больше среднего арифметического и удалить их из контейнера	Каждый элемент домножить на максимальный элемент контейнера

Анализ задачи

Задача 1 (vector Money)

1. Создаём вектор Money и вводим 5 сумм через push_back.
2. Находим минимальный элемент через min_element.
3. Вставляем минимальный элемент на указанную пользователем позицию через insert.
4. Вычисляем среднее арифметическое через for_each с лямбдой, удаляем элементы больше среднего через remove_if и erase.
5. Находим максимальный элемент через max_element, умножаем каждый элемент на максимальный через transform с функтором MultiplyByMax.
6. Сортируем вектор по возрастанию через sort, затем по убыванию через greater.
7. Ищем введённую пользователем сумму через find, считаем количество через count_if.

Задача 2 (priority_queue Money)

1. Создаём priority_queue с компаратором greater (очередь по возрастанию), вводим 5 сумм через push.
2. Извлекаем все элементы во временный вектор для выполнения операций.
3. Находим минимальный элемент, вставляем его на указанную позицию в векторе.
4. Вычисляем среднее арифметическое, удаляем элементы больше среднего.
5. Находим максимальный элемент, умножаем каждый элемент на максимальный через transform.
6. Очищаем исходную очередь и заново заполняем её из изменённого вектора.
7. Выводим итоговую очередь.

Задача 3 (multiset Money)

1. Создаём multiset Money и вводим 5 сумм через insert.

2. Находим минимальный элемент через `min_element`, копируем `multiset` во временный вектор для вставки по позиции, вставляем минимальный элемент, очищаем `multiset` и заполняем заново.
3. Вычисляем среднее арифметическое, копируем элементы не больше среднего в новый вектор через `remove_copy_if`, очищаем `multiset` и заполняем заново.
4. Находим максимальный элемент, умножаем каждый элемент на максимальный через `transform`, очищаем `multiset` и заполняем заново.
5. Выводим сортировку по возрастанию (`multiset` уже отсортирован) и по убыванию через `rbegin` и `rend`.
6. Ищем элемент через `find`, считаем количество через `count`.

Задача 4 (queue Money)

1. Создаём `queue Money` и вводим 5 сумм через `push`.
2. Извлекаем все элементы во временный вектор для выполнения операций.
3. Находим минимальный элемент, вставляем его на указанную позицию в векторе.
4. Вычисляем среднее арифметическое, удаляем элементы больше среднего через `remove_if` и `erase`.
5. Находим максимальный элемент, умножаем каждый элемент на максимальный через `transform`.
6. Сортируем вектор по возрастанию и убыванию.
7. Ищем элемент через `find`, считаем количество через `count_if`.

Задача 5 (deque Money)

1. Создаём `deque Money` и вводим 5 сумм через `push_back`.
2. Находим минимальный элемент через `min_element`, вставляем его на указанную позицию через `insert`.
3. Вычисляем среднее арифметическое, удаляем элементы больше среднего через `remove_if` и `erase`.
4. Находим максимальный элемент, умножаем каждый элемент на максимальный через `transform` с `back_inserter`.
5. Сортируем `deque` по возрастанию и убыванию.
6. Ищем элемент через `find`, считаем количество через `count_if`.

Код

```
#include <iostream>
#include <vector>
#include <queue>
#include <set>
#include <stack>
#include <list>
#include <algorithm>
#include <locale>
```

```
#include <functional>
```

```
#include <iterator>
```

```
using namespace std;
```

```
class Money {
```

```
private:
```

```
    long rubles;
```

```
    int kopeks;
```

```
public:
```

```
    Money() : rubles(0), kopeks(0) {}
```

```
    Money(long r, int k) : rubles(r), kopeks(k) {  
        normalize();  
    }
```

```
    Money(const Money& other) : rubles(other.rubles), kopeks(other.kopeks) {}
```

```
    ~Money() {}
```

```
    Money& operator=(const Money& other) {  
        if (this != &other) {  
            rubles = other.rubles;  
            kopeks = other.kopeks;  
        }  
        return *this;  
    }
```

```
    bool operator==(const Money& other) const {  
        return (rubles == other.rubles && kopeks == other.kopeks);  
    }
```

```
    bool operator!=(const Money& other) const {  
        return !(*this == other);  
    }
```

```
    bool operator>(const Money& other) const {  
        if (rubles != other.rubles) {  
            return rubles > other.rubles;  
        }  
        return kopeks > other.kopeks;  
    }
```

```
    bool operator<(const Money& other) const {
```

```

    if (rubles != other.rubles) {
        return rubles < other.rubles;
    }
    return kopeks < other.kopeks;
}

bool operator>=(const Money& other) const {
    return (*this > other) || (*this == other);
}

bool operator<=(const Money& other) const {
    return (*this < other) || (*this == other);
}

Money operator+(const Money& other) const {
    Money result;
    result.rubles = rubles + other.rubles;
    result.kopeks = kopeks + other.kopeks;
    result.normalize();
    return result;
}

Money operator*(const Money& other) const {
    double val1 = rubles + kopeks / 100.0;
    double val2 = other.rubles + other.kopeks / 100.0;
    double product = val1 * val2;
    long long total = (long long)(product * 100.0);
    Money result;
    if (total >= 0) {
        result.rubles = (long)(total / 100);
        result.kopeks = (int)(total % 100);
    }
    else {
        result.rubles = (long)(total / 100) - 1;
        result.kopeks = (int)(100 + (total % 100));
    }
    result.normalize();
    return result;
}

Money operator*(int multiplier) const {
    Money result;
    long long total = (long long)(rubles) * 100 + kopeks;
    total = total * multiplier;
    result.rubles = (long)(total / 100);

```

```

    result.kopeks = (int)(total % 100);
    if (result.kopeks < 0) {
        result.rubles--;
        result.kopeks += 100;
    }
    result.normalize();
    return result;
}

Money& operator+=(const Money& other) {
    rubles += other.rubles;
    kopeks += other.kopeks;
    normalize();
    return *this;
}

double toDouble() const {
    return (double)rubles + (double)kopeks / 100.0;
}

long getRubles() const {
    return rubles;
}

int getKopeks() const {
    return kopeks;
}

void normalize() {
    if (kopeks >= 100) {
        rubles += kopeks / 100;
        kopeks %= 100;
    }
    else if (kopeks < 0) {
        long newRubles = rubles + (kopeks / 100) - 1;
        int newKopeks = 100 + (kopeks % 100);
        if (newKopeks >= 100) {
            newRubles += newKopeks / 100;
            newKopeks %= 100;
        }
        rubles = newRubles;
        kopeks = newKopeks;
    }
}

```

```

friend istream& operator>>(istream& in, Money& m);
friend ostream& operator<<(ostream& out, const Money& m);
};

istream& operator>>(istream& in, Money& m) {
    cout << "Введите рубли: ";
    in >> m.rubles;
    cout << "Введите копейки: ";
    in >> m.kopeks;
    m.normalize();
    return in;
}

ostream& operator<<(ostream& out, const Money& m) {
    out << m.rubles << ",";
    if (m.kopeks < 10) {
        out << "0";
    }
    out << m.kopeks;
    return out;
}

void printMoney(const Money& m) {
    cout << m << " ";
}

struct MultiplyByMax {
    Money maxVal;
    MultiplyByMax(const Money& max) : maxVal(max) {}
    Money operator()(const Money& m) const {
        return m * maxVal;
    }
};

void task1() {
    cout << "\nЗАДАЧА 1" << endl;
    cout << "Контейнер: vector\nТип: Money\n" << endl;

    vector<Money> vec;

    cout << "Введите 5 сумм Money:" << endl;
    for (int i = 0; i < 5; i++) {
        Money m;
        cout << "\nСумма " << i + 1 << ":" << endl;
        cin >> m;
    }
}

```

```

    vec.push_back(m);
}

cout << "\nИсходный вектор: ";
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

Money minVal = *min_element(vec.begin(), vec.end());
cout << "\nМинимальный элемент: " << minVal << endl;

int pos;
cout << "Введите позицию для вставки минимального элемента (0-" <<
vec.size() << "): ";
cin >> pos;

if (pos >= 0 && pos <= (int)vec.size()) {
    vec.insert(vec.begin() + pos, minVal);
}

cout << "\nПосле добавления минимального элемента: ";
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

double sum = 0;
for_each(vec.begin(), vec.end(), [&sum](const Money& m) { sum +=
m.toDouble(); });
double avg = sum / vec.size();
cout << "\nСреднее арифметическое: " << avg << endl;

auto it = remove_if(vec.begin(), vec.end(), [avg](const Money& m) {
    return m.toDouble() > avg;
});
vec.erase(it, vec.end());

cout << "После удаления элементов больше среднего: ";
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

if (!vec.empty()) {
    Money maxVal = *max_element(vec.begin(), vec.end());
    cout << "\nМаксимальный элемент: " << maxVal << endl;

    vector<Money> multiplied(vec.size());
    transform(vec.begin(), vec.end(), multiplied.begin(),
MultiplyByMax(maxVal));
}

```



```

    vec = multiplied;

    cout << "После умножения каждого элемента на максимальный: ";
    for_each(vec.begin(), vec.end(), printMoney);
    cout << endl;
}

cout << "\nСортировка по возрастанию: ";
sort(vec.begin(), vec.end());
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

cout << "Сортировка по убыванию: ";
sort(vec.begin(), vec.end(), greater<Money>());
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

Money searchValue;
cout << "\nВведите сумму для поиска: ";
cin >> searchValue;

auto findIt = find(vec.begin(), vec.end(), searchValue);
if (findIt != vec.end()) {
    cout << "Элемент найден!" << endl;
}
else {
    cout << "Элемент не найден!" << endl;
}

int cnt = count_if(vec.begin(), vec.end(), [&searchValue](const Money& m) {
    return m == searchValue;
});
cout << "Количество таких элементов: " << cnt << endl;
}

void task2() {
    cout << "\nЗАДАЧА 2" << endl;
    cout << "Адаптер контейнера: priority_queue\nТип: Money\n" << endl;

    priority_queue<Money, vector<Money>, greater<Money>> pq;

    cout << "Введите 5 сумм Money:" << endl;
    for (int i = 0; i < 5; i++) {
        Money m;
        cout << "\nСумма " << i + 1 << ":" << endl;
    }
}

```

```

    cin >> m;
    pq.push(m);
}

cout << "\nИсходная очередь (по возрастанию):" << endl;
priority_queue<Money, vector<Money>, greater<Money>> temp = pq;
while (!temp.empty()) {
    cout << temp.top() << " ";
    temp.pop();
}
cout << endl;

vector<Money> vec;
temp = pq;
while (!temp.empty()) {
    vec.push_back(temp.top());
    temp.pop();
}

Money minVal = *min_element(vec.begin(), vec.end());
cout << "\nМинимальный элемент: " << minVal << endl;

int pos;
cout << "Введите позицию для вставки минимального элемента (0-" <<
vec.size() << "): ";
cin >> pos;

if (pos >= 0 && pos <= (int)vec.size()) {
    vec.insert(vec.begin() + pos, minVal);
}

cout << "\nПосле добавления минимального элемента: ";
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

double sum = 0;
for_each(vec.begin(), vec.end(), [&sum](const Money& m) { sum +=
m.toDouble(); });
double avg = sum / vec.size();
cout << "\nСреднее арифметическое: " << avg << endl;

auto it = remove_if(vec.begin(), vec.end(), [avg](const Money& m) {
    return m.toDouble() > avg;
});
vec.erase(it, vec.end());

```

```

cout << "После удаления элементов больше среднего: ";
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

if (!vec.empty()) {
    Money maxVal = *max_element(vec.begin(), vec.end());
    cout << "\nМаксимальный элемент: " << maxVal << endl;

    vector<Money> multiplied(vec.size());
    transform(vec.begin(), vec.end(), multiplied.begin(),
MultiplyByMax(maxVal));
    vec = multiplied;

    cout << "После умножения каждого элемента на максимальный: ";
    for_each(vec.begin(), vec.end(), printMoney);
    cout << endl;
}

while (!pq.empty()) pq.pop();
for (const auto& m : vec) {
    pq.push(m);
}

cout << "\nОчередь после всех операций: ";
temp = pq;
while (!temp.empty()) {
    cout << temp.top() << " ";
    temp.pop();
}
cout << endl;
}

void task3() {
    cout << "\nЗАДАЧА 3" << endl;
    cout << "Ассоциативный контейнер: multiset\nТип: Money\n" << endl;

    multiset<Money> ms;

    cout << "Введите 5 сумм Money:" << endl;
    for (int i = 0; i < 5; i++) {
        Money m;
        cout << "\nСумма " << i + 1 << ":" << endl;
        cin >> m;
        ms.insert(m);
    }
}

```

```

}

cout << "\nИсходное множество: ";
for_each(ms.begin(), ms.end(), printMoney);
cout << endl;

Money minVal = *min_element(ms.begin(), ms.end());
cout << "\nМинимальный элемент: " << minVal << endl;

int pos;
cout << "Введите позицию для вставки минимального элемента (0-" <<
ms.size() << "): ";
cin >> pos;

vector<Money> tempVec(ms.begin(), ms.end());
if (pos >= 0 && pos <= (int)tempVec.size()) {
    tempVec.insert(tempVec.begin() + pos, minVal);
}
ms.clear();
for (const auto& m : tempVec) {
    ms.insert(m);
}

cout << "\nПосле добавления минимального элемента: ";
for_each(ms.begin(), ms.end(), printMoney);
cout << endl;

double sum = 0;
for_each(ms.begin(), ms.end(), [&sum](const Money& m) { sum +=
m.toDouble(); });
double avg = sum / ms.size();
cout << "\nСреднее арифметическое: " << avg << endl;

tempVec.clear();
remove_copy_if(ms.begin(), ms.end(), back_inserter(tempVec), [avg](const
Money& m) {
    return m.toDouble() > avg;
});
ms.clear();
for (const auto& m : tempVec) {
    ms.insert(m);
}

cout << "После удаления элементов больше среднего: ";
for_each(ms.begin(), ms.end(), printMoney);

```

```

cout << endl;

if (!ms.empty()) {
    Money maxVal = *max_element(ms.begin(), ms.end());
    cout << "\nМаксимальный элемент: " << maxVal << endl;

    tempVec.clear();
    for (const auto& m : ms) {
        tempVec.push_back(m);
    }

    vector<Money> multiplied(tempVec.size());
    transform(tempVec.begin(), tempVec.end(), multiplied.begin(),
MultiplyByMax(maxVal));

    ms.clear();
    for (const auto& m : multiplied) {
        ms.insert(m);
    }

    cout << "После умножения каждого элемента на максимальный: ";
    for_each(ms.begin(), ms.end(), printMoney);
    cout << endl;
}

cout << "\nСортировка по возрастанию (multiset уже отсортирован): ";
for_each(ms.begin(), ms.end(), printMoney);
cout << endl;

cout << "Сортировка по убыванию: ";
for_each(ms.rbegin(), ms.rend(), printMoney);
cout << endl;

Money searchValue;
cout << "\nВведите сумму для поиска: ";
cin >> searchValue;

auto findIt = find(ms.begin(), ms.end(), searchValue);
if (findIt != ms.end()) {
    cout << "Элемент найден!" << endl;
}
else {
    cout << "Элемент не найден!" << endl;
}
}

```

```

int cnt = count(ms.begin(), ms.end(), searchValue);
cout << "Количество таких элементов: " << cnt << endl;
}

void task4() {
    cout << "\nЗАДАЧА 4" << endl;
    cout << "Адаптер контейнера: queue\nТип: Money\n" << endl;

    queue<Money> q;

    cout << "Введите 5 сумм Money:" << endl;
    for (int i = 0; i < 5; i++) {
        Money m;
        cout << "\nСумма " << i + 1 << ":" << endl;
        cin >> m;
        q.push(m);
    }

    cout << "\nИсходная очередь: ";
    queue<Money> tempQ = q;
    while (!tempQ.empty()) {
        cout << tempQ.front() << " ";
        tempQ.pop();
    }
    cout << endl;

    vector<Money> vec;
    tempQ = q;
    while (!tempQ.empty()) {
        vec.push_back(tempQ.front());
        tempQ.pop();
    }

    Money minVal = *min_element(vec.begin(), vec.end());
    cout << "\nМинимальный элемент: " << minVal << endl;

    int pos;
    cout << "Введите позицию для вставки минимального элемента (0-" <<
vec.size() << "): ";
    cin >> pos;

    if (pos >= 0 && pos <= (int)vec.size()) {
        vec.insert(vec.begin() + pos, minVal);
    }
}

```

```

cout << "\nПосле добавления минимального элемента: ";
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

double sum = 0;
for_each(vec.begin(), vec.end(), [&sum](const Money& m) { sum +=
m.toDouble(); });
double avg = sum / vec.size();
cout << "\nСреднее арифметическое: " << avg << endl;

auto it = remove_if(vec.begin(), vec.end(), [avg](const Money& m) {
    return m.toDouble() > avg;
});
vec.erase(it, vec.end());

cout << "После удаления элементов больше среднего: ";
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

if (!vec.empty()) {
    Money maxVal = *max_element(vec.begin(), vec.end());
    cout << "\nМаксимальный элемент: " << maxVal << endl;

    vector<Money> multiplied(vec.size());
    transform(vec.begin(), vec.end(), multiplied.begin(),
MultiplyByMax(maxVal));
    vec = multiplied;

    cout << "После умножения каждого элемента на максимальный: ";
    for_each(vec.begin(), vec.end(), printMoney);
    cout << endl;
}

cout << "\nСортировка по возрастанию: ";
sort(vec.begin(), vec.end());
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

cout << "Сортировка по убыванию: ";
sort(vec.begin(), vec.end(), greater<Money>());
for_each(vec.begin(), vec.end(), printMoney);
cout << endl;

Money searchValue;
cout << "\nВведите сумму для поиска: ";

```

```

cin >> searchValue;

auto findIt = find(vec.begin(), vec.end(), searchValue);
if (findIt != vec.end()) {
    cout << "Элемент найден!" << endl;
}
else {
    cout << "Элемент не найден!" << endl;
}

int cnt = count_if(vec.begin(), vec.end(), [&searchValue](const Money& m) {
    return m == searchValue;
});
cout << "Количество таких элементов: " << cnt << endl;
}

void task5() {
    cout << "\nЗАДАЧА 5" << endl;
    cout << "Контейнер: deque\nТип: Money\n" << endl;

    deque<Money> dq;

    cout << "Введите 5 сумм Money:" << endl;
    for (int i = 0; i < 5; i++) {
        Money m;
        cout << "\nСумма " << i + 1 << ":" << endl;
        cin >> m;
        dq.push_back(m);
    }

    cout << "\nИсходная deque: ";
    for_each(dq.begin(), dq.end(), printMoney);
    cout << endl;

    Money minVal = *min_element(dq.begin(), dq.end());
    cout << "\nМинимальный элемент: " << minVal << endl;

    int pos;
    cout << "Введите позицию для вставки минимального элемента (0-" <<
dq.size() << "): ";
    cin >> pos;

    if (pos >= 0 && pos <= (int)dq.size()) {
        dq.insert(dq.begin() + pos, minVal);
    }
}

```



```

cout << "\nПосле добавления минимального элемента: ";
for_each(dq.begin(), dq.end(), printMoney);
cout << endl;

double sum = 0;
for_each(dq.begin(), dq.end(), [&sum](const Money& m) { sum +=
m.toDouble(); });
double avg = sum / dq.size();
cout << "\nСреднее арифметическое: " << avg << endl;

auto it = remove_if(dq.begin(), dq.end(), [avg](const Money& m) {
    return m.toDouble() > avg;
});
dq.erase(it, dq.end());

cout << "После удаления элементов больше среднего: ";
for_each(dq.begin(), dq.end(), printMoney);
cout << endl;

if (!dq.empty()) {
    Money maxVal = *max_element(dq.begin(), dq.end());
    cout << "\nМаксимальный элемент: " << maxVal << endl;

    deque<Money> multiplied;
    transform(dq.begin(), dq.end(), back_inserter(multiplied),
MultiplyByMax(maxVal));
    dq = multiplied;

    cout << "После умножения каждого элемента на максимальный: ";
    for_each(dq.begin(), dq.end(), printMoney);
    cout << endl;
}

cout << "\nСортировка по возрастанию: ";
sort(dq.begin(), dq.end());
for_each(dq.begin(), dq.end(), printMoney);
cout << endl;

cout << "Сортировка по убыванию: ";
sort(dq.begin(), dq.end(), greater<Money>());
for_each(dq.begin(), dq.end(), printMoney);
cout << endl;

Money searchValue;

```

```

cout << "\nВведите сумму для поиска: ";
cin >> searchValue;

auto findIt = find(dq.begin(), dq.end(), searchValue);
if (findIt != dq.end()) {
    cout << "Элемент найден!" << endl;
}
else {
    cout << "Элемент не найден!" << endl;
}

int cnt = count_if(dq.begin(), dq.end(), [&searchValue](const Money& m) {
    return m == searchValue;
});
cout << "Количество таких элементов: " << cnt << endl;
}

int main() {
    setlocale(LC_ALL, "Russian");

    int choice;

    do {
        cout << "\nГЛАВНОЕ МЕНЮ" << endl;
        cout << "1. Задача 1 (vector<Money>)" << endl;
        cout << "2. Задача 2 (priority_queue<Money>)" << endl;
        cout << "3. Задача 3 (multiset<Money>)" << endl;
        cout << "4. Задача 4 (queue<Money>)" << endl;
        cout << "5. Задача 5 (deque<Money>)" << endl;
        cout << "0. Выход" << endl;
        cout << "Ваш выбор: ";
        cin >> choice;

        switch (choice) {
            case 1:
                task1();
                break;
            case 2:
                task2();
                break;
            case 3:
                task3();
                break;
            case 4:
                task4();

```

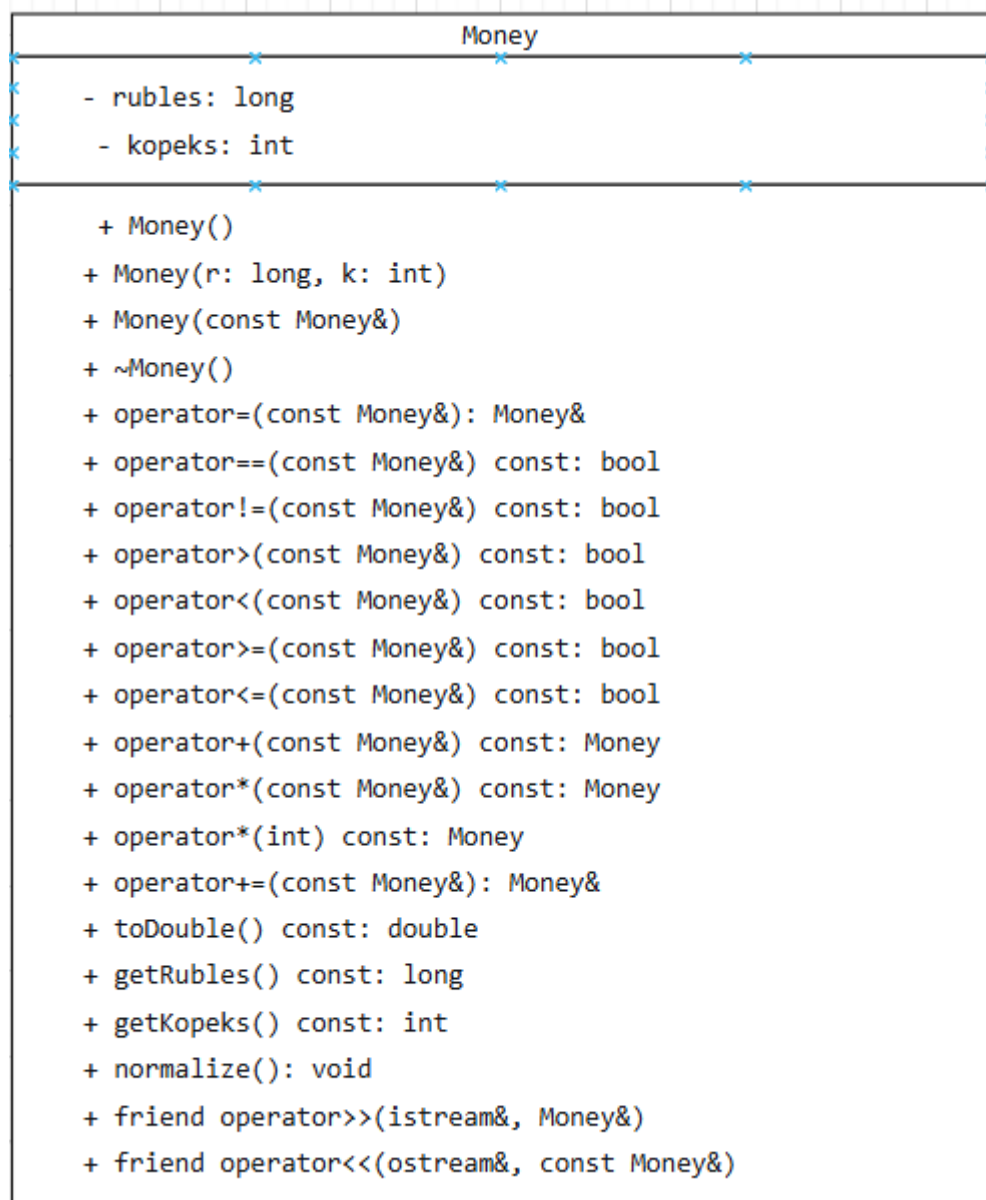
```

        break;
    case 5:
        task5();
        break;
    case 0:
        cout << "Выход из программы..." << endl;
        break;
    default:
        cout << "Неверный выбор!" << endl;
    }
} while (choice != 0);

return 0;
}

```

UML диаграмма



Вывод программы

```
ГЛАВНОЕ МЕНЮ
1. Задача 1 (vector<Money>)
2. Задача 2 (priority_queue<Money>)
3. Задача 3 (multiset<Money>)
4. Задача 4 (queue<Money>)
5. Задача 5 (deque<Money>)
0. Выход
Ваш выбор: 1

ЗАДАЧА 1
Контейнер: vector
Тип: Money

Введите 5 сумм Money:

Сумма 1:
Введите рубли: 30
Введите копейки: 60

Сумма 2:
Введите рубли: 2
Введите копейки: 40

Сумма 3:
Введите рубли: 30
Введите копейки: 30

Сумма 4:
Введите рубли: 20
Введите копейки: 30

Сумма 5:
Введите рубли: 23
Введите копейки: 23

Исходный вектор: 30,60 2,40 30,30 20,30 23,23

Минимальный элемент: 2,40
Введите позицию для вставки минимального элемента (0-5): 2

После добавления минимального элемента: 30,60 2,40 2,40 30,30 20,30 23,23

Среднее арифметическое: 18.205
После удаления элементов больше среднего: 2,40 2,40

Максимальный элемент: 2,40
После умножения каждого элемента на максимальный: 5,76 5,76

Сортировка по возрастанию: 5,76 5,76
Сортировка по убыванию: 5,76 5,76

Введите сумму для поиска: Введите рубли:
20
Введите копейки: 30
Элемент не найден!
Количество таких элементов: 0
```

```
ЗАДАЧА 2
Адаптер контейнера: priority_queue
Тип: Money

Введите 5 сумм Money:

Сумма 1:
Введите рубли: 30
Введите копейки: 34

Сумма 2:
Введите рубли: 7
Введите копейки: 34

Сумма 3:
Введите рубли: 4
Введите копейки: 34

Сумма 4:
Введите рубли: 3
Введите копейки: 9

Сумма 5:
Введите рубли: 40
Введите копейки: 3

Исходная очередь (по возрастанию):
3,09 4,34 7,34 30,34 40,03

Минимальный элемент: 3,09
Введите позицию для вставки минимального элемента (0-5): 3

После добавления минимального элемента: 3,09 4,34 7,34 3,09 30,34 40,03

Среднее арифметическое: 14.705
После удаления элементов больше среднего: 3,09 4,34 7,34 3,09

Максимальный элемент: 7,34
После умножения каждого элемента на максимальный: 22,68 31,85 53,87 22,68

Очередь после всех операций: 22,68 22,68 31,85 53,87
```

```
ЗАДАЧА 3
Ассоциативный контейнер: multiset
Тип: Money

Введите 5 сумм Money:

Сумма 1:
Введите рубли: 30
Введите копейки: 3

Сумма 2:
Введите рубли: 20
Введите копейки: 2

Сумма 3:
Введите рубли: 12
Введите копейки: 3

Сумма 4:
Введите рубли: 30
Введите копейки: 50

Сумма 5:
Введите рубли: 50
Введите копейки: 30

Исходное множество: 12,03 20,02 30,03 30,50 50,30

Минимальный элемент: 12,03
Введите позицию для вставки минимального элемента (0-5): 2

После добавления минимального элемента: 12,03 12,03 20,02 30,03 30,50 50,30

Среднее арифметическое: 25.8183
После удаления элементов больше среднего: 12,03 12,03 20,02

Максимальный элемент: 20,02
После умножения каждого элемента на максимальный: 240,84 240,84 400,80

Сортировка по возрастанию (multiset уже отсортирован): 240,84 240,84 400,80
Сортировка по убыванию: 400,80 240,84 240,84

Введите сумму для поиска: Введите рубли: 12
Введите копейки: 3
Элемент не найден!
Количество таких элементов: 0
```

ЗАДАЧА 4

Адаптер контейнера: queue

Тип: Money

Введите 5 сумм Money:

Сумма 1:

Введите рубли: 2

Введите копейки: 23

Сумма 2:

Введите рубли: 23

Введите копейки: 4

Сумма 3:

Введите рубли: 4

Введите копейки: 8

Сумма 4:

Введите рубли: 4

Введите копейки: 0

Сумма 5:

Введите рубли: 60

Введите копейки: 56

Исходная очередь: 2,23 23,04 4,08 4,00 60,56

Минимальный элемент: 2,23

Введите позицию для вставки минимального элемента (0-5): 5

После добавления минимального элемента: 2,23 23,04 4,08 4,00 60,56 2,23

Среднее арифметическое: 16.0233

После удаления элементов больше среднего: 2,23 4,08 4,00 2,23

Максимальный элемент: 4,08

После умножения каждого элемента на максимальный: 9,09 16,64 16,32 9,09

Сортировка по возрастанию: 9,09 9,09 16,32 16,64

Сортировка по убыванию: 16,64 16,32 9,09 9,09

Введите сумму для поиска: Введите рубли: 4

Введите копейки: 0

Элемент не найден!

Количество таких элементов: 0

ГЛАВНОЕ МЕНЮ

1. Задача 1 (vector<Money>)
 2. Задача 2 (priority_queue<Money>)
 3. Задача 3 (multiset<Money>)
 4. Задача 4 (queue<Money>)
 5. Задача 5 (deque<Money>)
 0. Выход
- Ваш выбор: 5

ЗАДАЧА 5

Контейнер: deque

Тип: Money

Введите 5 сумм Money:

Сумма 1:

Введите рубли: 1

Введите копейки: 2

Сумма 2:

Введите рубли: 3

Введите копейки: 4

Сумма 3:

Введите рубли: 2

Введите копейки: 3

Сумма 4:

Введите рубли: 4

Введите копейки: 2

Сумма 5:

Введите рубли: 4

Введите копейки: 23

Исходная deque: 1,02 3,04 2,03 4,02 4,23

Минимальный элемент: 1,02

Введите позицию для вставки минимального элемента (0-5):

2

После добавления минимального элемента: 1,02 3,04 1,02 2,03 4,02 4,23

Среднее арифметическое: 2.56

После удаления элементов больше среднего: 1,02 1,02 2,03

Максимальный элемент: 2,03

После умножения каждого элемента на максимальный: 2,07 2,07 4,12

Сортировка по возрастанию: 2,07 2,07 4,12

Сортировка по убыванию: 4,12 2,07 2,07

Введите сумму для поиска: Введите рубли: 2

Введите копейки: 2

Элемент не найден!